

Extraindo dados de um repositório - Aula 5

🕒 Criado em	9 de setembro de 2026 14:53
🏷️ Tags	

1. Visão geral da aula

Na aula anterior estudamos conceitos relacionados a:

- esforço;
- homem-mês;
- tamanho das equipes;
- comunicação;
- produtividade;
- manutenção;
- modularização;
- evolução do software.

Nesta aula, esses conceitos começam a ser colocados em prática.

Em vez de analisar um projeto apenas "olhando para o código", podemos utilizar um repositório Git para extrair informações objetivas sobre o desenvolvimento.

Por exemplo:

- Quantas pessoas contribuíram?
- Quem realizou mais commits?
- Quais arquivos mudam com maior frequência?
- Qual é a idade do código?
- Quando determinado desenvolvedor contribuiu?
- Como a quantidade de commits evoluiu ao longo do tempo?

- Quais funções possuem maior complexidade?
- Existem partes do código que podem ser mais difíceis de manter?

A ideia geral é:

| Extrair dados → calcular métricas → interpretar os resultados.

2. Por que analisar um repositório?

Um repositório Git não serve apenas para armazenar código.

Ele também mantém um histórico do desenvolvimento.

Cada commit pode fornecer informações sobre:

- autor;
- data;
- arquivos modificados;
- evolução do código;
- alterações realizadas;
- relacionamento entre versões.

Assim, o histórico do Git pode ser utilizado como uma fonte de dados para estudar o projeto.

Podemos pensar:

```
Código + Histórico Git
    ↓
  Dados brutos
    ↓
  Métricas
    ↓
  Análise
    ↓
Conclusões
```

Isso é especialmente importante em Engenharia de Programas porque permite analisar o desenvolvimento de maneira mais objetiva.

3. Primeiro passo: obter um repositório

O primeiro comando utilizado foi:

```
git clone https://github.com/flyrell/axios-auth-refresh
```

O que ele faz?

`git clone` cria uma cópia local de um repositório remoto.

Nesse caso:

```
GitHub
  ↓
axios-auth-refresh
  ↓
Computador / Colab
```

Depois do clone, é possível analisar localmente:

- código;
- branches;
- tags;
- commits;
- autores;
- histórico.

4. Por que usar um repositório real?

Porque um projeto real possui um histórico de desenvolvimento.

Se criássemos um projeto vazio agora, teríamos poucos dados.

Em um projeto existente, podemos observar:

- vários desenvolvedores;
- vários commits;
- diferentes versões;

- evolução durante meses ou anos;
- arquivos modificados repetidamente.

Isso fornece dados para análise.

5. Entrando na pasta do projeto

Depois de clonar o projeto, precisamos trabalhar dentro da pasta:

```
cd axios-auth-refresh
```

No Colab, o comando aparece como:

```
%cd axios-auth-refresh
```

A partir desse momento, os comandos Git são executados considerando aquele repositório.

6. `ls` e `pwd`

O material também utiliza:

```
ls
```

e:

```
pwd
```

```
ls
```

Lista os arquivos e diretórios existentes.

```
pwd
```

Mostra o diretório atual.

Em um ambiente Linux/Colab:

```
pwd  
↓
```

```
/content/axios-auth-refresh
```

Isso é útil para confirmar:

| "Estou realmente dentro do repositório que quero analisar?"

7. Analisando branches locais

O comando:

```
git branch -vv
```

mostra informações sobre as branches locais.

O objetivo aqui é descobrir:

- quais branches existem localmente;
- qual branch está atualmente selecionada;
- qual commit cada branch aponta;
- informações sobre rastreamento de branches remotas.

O `-vv` significa que o Git mostra informações adicionais.

8. Branches locais e remotas

Para visualizar branches locais e remotas:

```
git branch -a -vv
```

O `-a` significa:

| mostrar todas as branches, incluindo as remotas.

Assim podemos ter algo semelhante a:

```
* master
remotes/origin/master
remotes/origin/develop
```

Isso permite observar a estrutura de desenvolvimento existente no projeto.

9. Tags

Outro conceito importante são as **tags**.

O comando:

```
git tag
```

lista as tags existentes no repositório.

Tags normalmente são utilizadas para identificar versões específicas.

Por exemplo:

```
v1.0.0  
v2.0.0  
v3.3.5
```

Podemos pensar em uma tag como uma marcação:

| "Esta versão específica do código é importante."

10. Por que tags são importantes para análise?

Imagine que queremos estudar:

| "Como o software estava na versão 3.3.5?"

Podemos acessar exatamente aquela versão.

O comando utilizado no material é:

```
git checkout v3.3.5
```

Isso coloca o repositório naquele estado específico.

11. Tag não é simplesmente uma branch

Essa distinção é importante.

Branch

Representa uma linha de desenvolvimento que pode continuar recebendo commits.

Tag

Normalmente identifica um ponto específico do histórico.

Por exemplo:

```
A — B — C — D — E
           ↑
        v3.3.5
```

A tag aponta para aquele ponto específico.

12. Voltando para a branch

Depois de analisar a versão:

```
git switch master
```

Também poderia ser utilizado:

```
git checkout master
```

A diferença é que `git switch` foi criado especificamente para operações relacionadas a branches, enquanto `git checkout` possui responsabilidades mais amplas.

Para estudo, o mais importante é entender:

Você acessou uma versão específica e depois voltou para a linha principal de desenvolvimento.

13. Descobrimos o repositório remoto

O comando:

```
git remote -v
```

mostra os repositórios remotos associados.

Um resultado pode ser parecido com:

```
origin https://github.com/flyrell/axios-auth-refresh  
origin https://github.com/flyrell/axios-auth-refresh
```

Normalmente existe um remoto chamado:

```
origin
```

que representa o repositório de onde o projeto foi obtido.

14. Histórico de commits

Agora começa uma das partes mais importantes da prática.

O comando:

```
git log --graph --decorate --all
```

permite visualizar o histórico do repositório.

Os parâmetros acrescentam informações:

```
git log
```

Mostra os commits.

```
-graph
```

Representa visualmente a estrutura de branches e merges.

```
-decorate
```

Mostra referências como:

- branches;
- tags;
- HEAD.

-all

Considera todas as referências disponíveis.

15. Por que o histórico é importante?

Porque o histórico conta a história do desenvolvimento.

Por exemplo:

```
A — B — C — D
          \
          E — F
```

Podemos identificar:

- desenvolvimento paralelo;
- criação de branches;
- merges;
- versões;
- evolução do projeto.

Isso transforma o Git em uma fonte de informações para análise.

16. Descobrimos os contribuidores

O comando apresentado é:

```
git log --format='%aN' | sort -u | tee users.txt
```

Esse comando pode parecer assustador inicialmente, mas podemos quebrá-lo.

git log

Obtém o histórico de commits.

```
-format='%aN'
```

Pede ao Git para mostrar o nome do autor de cada commit.

O `%aN` representa o nome do autor.

Assim podemos obter:

```
Ana
João
Maria
Ana
João
Ana
```

17. O `sort`

Depois temos:

```
| sort
```

O `|` é um **pipe**.

Ele pega a saída do comando anterior e envia para o próximo.

Então:

```
git log
↓
nomes
↓
sort
↓
nomes ordenados
```

18. O `uniq -u`

Depois:

```
| uniq -u
```

A intenção da combinação `sort` + `uniq` é eliminar repetições.

Porém, existe uma observação técnica importante:

O comportamento mais comum para obter valores únicos seria:

```
sort -u
```

ou:

```
sort | uniq
```

O `uniq -u`, especificamente, mantém apenas linhas que **não possuem repetição**.

Portanto, o comando da aula pode não produzir exatamente uma lista de todos os contribuidores se um mesmo autor aparecer várias vezes.

Para simplesmente obter os nomes distintos, uma forma mais adequada seria:

```
git log --format='%aN' | sort -u
```

Essa é uma boa pegadinha para entender.

19. O `tee`

No final aparece:

```
| tee users.txt
```

O `tee` permite:

| mostrar o resultado na tela e, simultaneamente, salvá-lo em um arquivo.

Então:

```
resultado  
|→ tela
```

↳ users.txt

Isso é útil quando queremos preservar os dados para análise posterior.

20. Arquivos mais ativos

O próximo comando:

```
git log --pretty=format: --name-only | sort | uniq -c | sort -nr | head -10 | tee file
```

tem uma finalidade interessante:

Descobrir quais arquivos aparecem com maior frequência no histórico de alterações.

Isso pode ajudar a identificar partes do sistema que sofrem muitas modificações.

21. Entendendo o comando por partes

git log

Obtém o histórico.

-pretty=format:

Remove as informações normais dos commits.

A intenção é deixar principalmente os nomes dos arquivos.

-name-only

Mostra apenas os nomes dos arquivos alterados.

Depois:

sort

ordena os nomes.

Depois:

```
uniq -c
```

conta quantas vezes cada nome aparece.

Depois:

```
sort -nr
```

ordena numericamente em ordem decrescente.

Depois:

```
head -10
```

pega os 10 primeiros.

22. O que essa métrica pode indicar?

Imagine que apareça:

```
120 src/api.js
95  src/auth.js
70  src/config.js
20  README.md
```

Isso significa que `src/api.js` apareceu em muito mais alterações do histórico.

Isso pode indicar:

- arquivo importante;
- arquivo central;
- arquivo muito utilizado;
- parte do sistema que sofre muitas mudanças;
- possível ponto de acoplamento.

Mas cuidado:

Muita alteração não significa automaticamente que o arquivo é ruim.

Pode ser simplesmente um arquivo central do sistema.

23. Arquivos muito grandes

O material utiliza:

```
find src_py -name '*.py' -type f -exec wc -l {} + | sort -rn | head -n 10
```

O objetivo é encontrar arquivos Python com muitas linhas.

24. Entendendo `find`

```
find src_py
```

procura arquivos dentro de `src_py`.

Depois:

```
-name '*.py'
```

restringe a arquivos Python.

E:

```
-type f
```

significa:

| procurar arquivos regulares.

25. Contando linhas

A parte:

```
wc -l
```

conta linhas.

`wc` significa "word count", mas com `-l` estamos pedindo a contagem de linhas.

Então:

```
arquivo.py → 850 linhas
```

26. Ordenando os maiores arquivos

Depois:

```
sort -rn
```

ordena numericamente em ordem reversa.

Assim, os maiores aparecem primeiro.

Finalmente:

```
head -n 10
```

mostra os 10 primeiros.

Resultado:

| Os 10 arquivos Python com maior quantidade de linhas.

27. Por que tamanho do arquivo importa?

Arquivos muito grandes podem ser sinais de:

- muitas responsabilidades concentradas;
- dificuldade de compreensão;
- baixa modularização;
- maior dificuldade de manutenção.

Mas novamente:

| **Um arquivo grande não é automaticamente um arquivo ruim.**

É uma métrica que precisa ser interpretada junto com outras informações.

O próprio material sugere ferramentas especializadas, como `cloc`, para análise de tamanho de código.

28. Medindo a idade do código

O comando:

```
git log --format="%aD" | tail -1
```

obtem datas dos commits e pega a última informação apresentada pelo histórico.

A ideia é identificar a data mais antiga registrada no histórico, permitindo estimar há quanto tempo aquele código/projeto possui histórico de desenvolvimento.

O `%aD` representa a data do autor em um formato legível.

29. Por que medir a idade do código?

A idade do código pode ajudar a responder perguntas como:

- Há quanto tempo esse projeto existe?
- Existem partes muito antigas?
- O código recebe manutenção constantemente?
- O projeto está ativo?
- Uma parte do sistema foi criada há muitos anos e quase não foi alterada?

Isso começa a conectar a aula com o conceito de:

envelhecimento do software.

30. Tempo de contribuição de um autor

O material também apresenta:

```
git log --author "Alex Malkevich" --format="%aD" | tail -1
```


A diferença é que agora o histórico é filtrado por autor.

O Git procura commits atribuídos à pessoa indicada.

Assim podemos investigar:

| "Desde quando esse autor possui contribuições registradas nesse projeto?"

31. Estatísticas por usuário

Outro comando:

```
git shortlog -sn --email --no-merges
```

Esse comando é muito interessante para estatísticas de contribuição.

git shortlog

Resume o histórico por autor.

s

Mostra a quantidade de commits.

n

Ordena numericamente.

-email

Inclui o e-mail.

-no-merges

Exclui commits de merge.

Assim podemos obter algo parecido com:

```
150 autor1@email.com
90  autor2@email.com
30  autor3@email.com
```

Isso fornece uma visão da distribuição dos commits.

32. Cuidado: número de commits não é produtividade

Esse é um ponto extremamente importante.

Não devemos concluir:

"Pessoa A fez 150 commits, então é 5 vezes mais produtiva que Pessoa B, que fez 30."

Isso seria uma interpretação incorreta.

Um commit pode conter:

- uma alteração pequena;
- uma grande funcionalidade;
- uma correção;
- uma documentação;
- uma mudança de configuração.

Portanto:

Quantidade de commits é uma métrica de atividade, não uma medida completa de produtividade.

Esse cuidado é muito importante em Engenharia de Programas.

33. Métrica por unidade de tempo

O material instala:

```
pip install git-bars
```

e depois utiliza:

```
git-bars -p month -a '2022-01-01'
```

A ferramenta `git-bars` permite analisar a atividade do repositório ao longo do tempo.

O parâmetro:

```
-p month
```

indica que a análise será agrupada por mês.

O:

```
-a '2022-01-01'
```

define uma data inicial para a análise.

34. Por que agrupar por tempo?

Porque uma lista total de commits não mostra a evolução.

Imagine:

```
Janeiro: 10 commits  
Fevereiro: 50 commits  
Março: 90 commits  
Abril: 20 commits
```

Agora conseguimos observar:

- períodos de maior atividade;
- períodos de menor atividade;
- crescimento;
- redução;
- possíveis ciclos de desenvolvimento.

Podemos visualizar isso em gráficos.

35. Commits por autor e tempo

O material também mostra:

```
git-bars -p month -a '2022-01-01' -u "<e-mail do autor>"
```

Agora a análise é filtrada por autor.

Assim podemos investigar:

Como a atividade de determinado desenvolvedor variou ao longo dos meses?

Isso é muito mais informativo do que olhar apenas para o total de commits.

36. O que estamos fazendo até aqui?

Até este ponto, a aula está ensinando a extrair **métricas do histórico do projeto**.

Podemos resumir:

Pergunta	Métrica/comando
Quais branches existem?	<code>git branch</code>
Quais versões existem?	<code>git tag</code>
Como o projeto evoluiu?	<code>git log</code>
Quem contribuiu?	<code>git log</code> / <code>shortlog</code>
Quem fez mais commits?	<code>git shortlog</code>
Quais arquivos mudam mais?	<code>git log --name-only</code>
Quais arquivos são maiores?	<code>find</code> + <code>wc -l</code>
Qual a idade do projeto?	<code>git log</code> + datas
Como a atividade varia no tempo?	<code>git-bars</code>

Mas agora a aula muda de nível.

Até aqui analisamos principalmente:

o histórico do desenvolvimento.

Agora vamos analisar:

a estrutura do código.

37. Complexidade Ciclomática

A **Complexidade Ciclomática** é uma métrica criada por **Thomas McCabe**.

Ela procura medir a quantidade de caminhos independentes existentes no fluxo de uma função ou módulo.

A ideia intuitiva é:

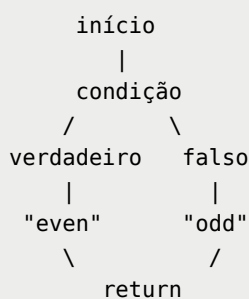
Quanto mais caminhos de execução existem, maior tende a ser a dificuldade de entender, testar e modificar aquela função.

38. Exemplo simples

Considere:

```
def isEven(n):  
    if (n % 2) == 0:  
        answer = 'even'  
    else:  
        answer = 'odd'  
  
    return answer
```

Existem dois caminhos principais:



Portanto, a função possui complexidade ciclomática:

$M = 2$

Uma maneira prática de chegar a esse resultado é:

número de decisões + 1

Existe:

```
1 if
```

Logo:

$1+1=2$ $1 + 1 = 2$

39. A fórmula formal

A fórmula apresentada no material é:

$$M = E - N + 2P$$

Onde:

M

Complexidade ciclomática.

E

Número de arestas (*edges*) do grafo.

N

Número de nós (*nodes*).

P

Número de componentes conexos.

Quando estamos analisando uma única função:

$$P=1 \quad P = 1$$

Então:

$$M = E - N + 2 \quad M = E - N + 2$$

40. O que são nós e arestas?

Para calcular a complexidade pela fórmula, podemos representar o fluxo da função como um grafo.

Nó

Representa um ponto do fluxo.

Por exemplo:

```
início  
condição  
ação  
fim
```

Aresta

Representa uma ligação entre dois nós.

Por exemplo:

```
A → B
```

significa que o fluxo pode passar de A para B.

41. O que significa "caminho independente"?

Um caminho independente é um caminho de execução que introduz uma nova possibilidade de fluxo.

No exemplo:

```
if condicao:  
    A  
else:  
    B
```

temos dois caminhos:

```
condição → A
```

e:

condição → B

Esses caminhos representam comportamentos diferentes que precisam ser considerados.

42. Regra prática

Para códigos simples, o material apresenta uma regra muito útil:

| Conte as condições e adicione 1.

Por exemplo:

```
if a:  
    ...
```

Temos:

1 decisão

Então:

$$M=1+1=2M = 1 + 1 = 2$$

43. Dois **if**

Imagine:

```
if a:  
    ...  
  
if b:  
    ...
```

Temos duas decisões.

Logo, pela regra simplificada:

$$M=2+1M = 2 + 1$$

$$M=3M = 3$$

44. Três condições

Se temos:

```
if a:
    ...

if b:
    ...

if c:
    ...
```

Então:

$$M=3+1M = 3 + 1$$

$$M=4M = 4$$

45. E **if** com **and** ?

Aqui precisamos ter um pouco mais de cuidado.

Considere:

```
if a and b:
    ...
```

Dependendo da convenção da ferramenta e da forma como a expressão é analisada, operadores lógicos podem contribuir para a complexidade.

Por isso, para análise real, é melhor utilizar uma ferramenta como o **Radon**, em vez de depender apenas da regra simplificada apresentada em aula.

Esse é um ponto importante:

┃ A regra "número de condições + 1" é uma aproximação didática.

46. Por que complexidade ciclomática importa?

Porque funções com muitos caminhos podem exigir:

- mais testes;
- mais casos de teste;
- maior esforço para compreender;
- maior cuidado ao modificar;
- maior risco de introduzir erros.

Imagine:

```
Função A  
Complexidade = 2
```

e:

```
Função B  
Complexidade = 15
```

A segunda possui uma estrutura de fluxo muito mais complexa.

Isso não significa automaticamente:

| "A função B está errada."

Mas significa:

| "Essa função merece atenção."

47. Complexidade não é sinônimo de quantidade de linhas

Esse é outro conceito importante.

Considere:

```
def exemplo():  
    linha1  
    linha2  
    linha3  
    ...  
    linha100
```

Pode ter 100 linhas, mas poucos caminhos.

Outra função pode possuir apenas 20 linhas:

```
if a:  
    if b:  
        if c:  
            ...  
        else:  
            ...  
    else:  
        ...  
else:  
    ...
```

Ela pode possuir muito mais caminhos.

Portanto:

Tamanho do código e complexidade do fluxo são métricas diferentes.

48. Limitações da Complexidade Ciclomática

Essa é uma das partes mais importantes da aula.

O material apresenta diferentes funções para mostrar que:

Uma métrica isolada não consegue explicar completamente a qualidade do código.

Considere:

```
def fn1(a, b, c):  
    if a:  
        if b:  
            if c:  
                pass
```

Existe uma estrutura aninhada.

Agora pense em outra forma de escrever a mesma lógica usando retornos antecipados.

Por exemplo:

```
def fn3(a, b, c):  
    if not a:  
        return  
  
    if not b:  
        return  
  
    if not c:  
        return
```

As duas estruturas possuem características diferentes.

49. O que isso demonstra?

Uma métrica pode fornecer um número.

Mas o número sozinho não conta toda a história.

É necessário observar:

- estrutura;
- legibilidade;
- modularização;
- responsabilidades;
- dependências;

- testes;
- contexto.

Portanto:

Métricas devem apoiar a análise humana, e não substituir a análise humana.

50. Uma função pode ter complexidade aceitável e ainda ser ruim

Imagine uma função:

```
def processar():  
    ...
```

Ela possui complexidade baixa.

Isso não significa necessariamente que ela esteja bem projetada.

Ela pode:

- ter uma responsabilidade enorme;
- acessar banco de dados diretamente;
- manipular arquivos;
- enviar e-mails;
- validar dados;
- controlar regras de negócio;
- possuir código duplicado.

Ou seja:

Complexidade ciclomática mede uma dimensão específica do software.

Não mede tudo.

51. Ferramenta Radon

Para automatizar a análise em Python, a aula apresenta o **Radon**.

Instalação:

```
pip install radon
```

Depois podemos consultar ajuda:

```
radon --help
```

E especificamente a análise de complexidade:

```
radon cc --help
```

52. **radon cc**

O comando principal é:

```
radon cc
```

O **cc** significa:

| Cyclomatic Complexity.

Podemos executar:

```
radon cc src_py
```

para analisar os arquivos Python daquele diretório.

53. A opção **a**

O material utiliza:

```
radon cc src_py -a
```

O **-a** solicita que o Radon apresente a complexidade média ao final da análise.

Assim, além de vermos funções individualmente, podemos ter uma visão geral.

54. Classificação de complexidade

O Radon também apresenta letras associadas aos níveis de complexidade.

De maneira geral, a escala utiliza classificações de:

```
A  
B  
C  
D  
E  
F
```

Sendo A uma classificação de menor complexidade e F uma classificação de maior complexidade.

O objetivo não é simplesmente:

| "A = código bom; F = código ruim."

É uma forma de sinalizar pontos que merecem atenção.

55. A opção **nc**

O material apresenta:

```
radon cc src_py -a -nc
```

A opção **-nc** faz o Radon mostrar apenas resultados classificados como **C ou pior**, conforme a descrição utilizada na aula.

Isso é útil quando queremos procurar:

| "Quais partes do código possuem complexidade que merece investigação?"

Em um projeto grande, isso evita analisar manualmente todas as funções de baixa complexidade.

56. Tipos de blocos

Na saída do Radon, podem aparecer letras indicando o tipo de elemento analisado.

Por exemplo:

```
F
```

representa:

| Function — função.

Outras classificações podem indicar:

```
M
```

| Method — método.

E:

```
C
```

| Class — classe.

Isso ajuda a interpretar exatamente qual elemento possui determinada métrica.

57. Exemplo conceitual de saída

Imagine que o Radon mostre algo semelhante a:

```
function_a - A
function_b - B
function_c - D
```

Podemos interpretar:

```
function_a
↓
baixa complexidade
```



```
function_b
↓
complexidade moderada

function_c
↓
maior complexidade
```

A função C merece uma análise mais cuidadosa.

58. O objetivo não é "zerar" a complexidade

Esse é um erro de interpretação comum.

Não devemos pensar:

▮ "Precisamos deixar toda função com complexidade 1."

Isso pode levar a uma simplificação artificial do código.

O objetivo é:

▮ **Identificar complexidade excessiva e avaliar se ela pode ser reduzida de maneira saudável.**

Por exemplo, uma função que naturalmente precisa de algumas decisões não deve ser dividida em dezenas de funções apenas para diminuir uma métrica.

59. Como reduzir complexidade?

Algumas estratégias incluem:

Modularização

Dividir responsabilidades.

Funções menores

Separar operações distintas.

Evitar condicionais excessivamente aninhadas

Estruturas muito profundas podem ser difíceis de entender.

Retornos antecipados

Em determinados casos, podem reduzir níveis de aninhamento.

Separação de responsabilidades

Uma função deve ter uma responsabilidade bem definida.

60. Relação com a aula anterior

Agora conseguimos conectar diretamente as duas aulas.

Na aula anterior vimos:

| Software precisa ser fácil de manter.

Agora aprendemos uma forma de investigar:

| Quais partes do código podem ser mais difíceis de manter?

A complexidade ciclomática é uma dessas ferramentas.

61. Relação com o envelhecimento do software

O professor também indica que a próxima aula vai relacionar:

- envelhecimento;
- métricas;
- repositórios reais;
- gráficos;
- evolução do código.

Isso cria uma linha de raciocínio:

```
Histórico Git
  ↓
Dados do desenvolvimento
  ↓
Métricas
  ↓
Identificação de padrões
```

↓
Análise da evolução
↓
Manutenção e envelhecimento

62. O verdadeiro objetivo da prática

A prática não é simplesmente aprender comandos do Git.

O objetivo maior é aprender a:

extrair informações de um projeto real para produzir evidências sobre sua evolução e características.

Git fornece o histórico.

Ferramentas como Radon analisam o código.

Depois, o engenheiro interpreta os resultados.

63. Tipos de métricas vistos na aula

Podemos separar as métricas em categorias.

Métricas do histórico

Exemplos:

- quantidade de commits;
- número de contribuidores;
- commits por autor;
- commits por período;
- arquivos mais modificados;
- idade do código.

Métricas estruturais

Exemplos:

- quantidade de linhas;

- complexidade ciclomática;
 - tamanho de arquivos;
 - complexidade de funções.
-

64. Por que combinar métricas?

Imagine que um arquivo tenha:

```
1.500 linhas
```

Isso chama atenção.

Mas agora descobrimos:

```
Complexidade baixa  
Poucas alterações  
Poucos dependentes
```

Talvez o tamanho seja justificável.

Agora imagine:

```
1.500 linhas  
+  
complexidade alta  
+  
muitas alterações  
+  
muitos módulos dependentes
```

Esse arquivo merece muito mais atenção.

Portanto:

| Métricas tornam-se mais úteis quando analisadas em conjunto.

65. Exemplo de análise profissional

Imagine que uma análise produza:

Arquivo: auth.py

Linhas: 900

Alterações: 180

Complexidade média: alta

Contribuidores: 8

Não devemos simplesmente dizer:

| "auth.py é ruim."

Uma análise mais adequada seria:

| "O arquivo possui grande tamanho, alta frequência de alterações, participação de diversos contribuidores e complexidade elevada. Esses fatores indicam que ele pode representar um ponto de manutenção relevante e merece investigação arquitetural."

Essa diferença entre **medir** e **interpretar** é essencial.

66. Métrica não é diagnóstico

Essa frase vale a pena memorizar:

| **Uma métrica é um indicador, não um diagnóstico completo.**

Por exemplo:

Muitos commits

não significa:

Pessoa muito produtiva

Da mesma forma:

Muitas linhas

não significa:

Código ruim

E:

Complexidade alta

não significa automaticamente:

Código incorreto

O contexto é necessário.

67. O que o Git está fornecendo?

Podemos pensar no Git como uma espécie de histórico do projeto:

```
Commit 1
  ↓
Commit 2
  ↓
Commit 3
  ↓
Commit 4
  ↓
...
```

Cada commit contém informações que podem ser utilizadas na análise.

Assim podemos extrair:

```
Quem?
Quando?
O quê?
Onde?
Com que frequência?
```

68. E o Radon?

O Radon olha para outra dimensão:

```
Código
↓
Estrutura
↓
Decisões
↓
Caminhos
↓
Complexidade
```

Portanto:

```
Git
↓
Histórico do desenvolvimento

Radon
↓
Características do código
```

Combinados:

```
Histórico + Código
↓
Métricas
↓
Engenharia
↓
Decisão
```

69. O que provavelmente será cobrado?

Eu daria bastante atenção aos seguintes pontos.

Git

Você deve saber explicar:

- `git clone` ;
- `git branch` ;

- `git branch -a ;`
- `git tag ;`
- `git checkout ;`
- `git switch ;`
- `git remote -v ;`
- `git log ;`
- `git shortlog .`

Não precisa apenas decorar os comandos.

Saiba:

| Qual pergunta cada comando ajuda a responder?

70. Pipes no terminal

Também vale entender:

|

O pipe conecta comandos.

Exemplo:

```
comando1 | comando2
```

Significa:

| A saída do comando 1 será utilizada como entrada do comando 2.

Isso permite construir pequenas etapas de processamento.

Por exemplo:

```
git log
  ↓
sort
  ↓
uniq
```


↓
sort
↓
head

Cada comando transforma os dados para a próxima etapa.

71. Complexidade Ciclomática

É muito importante saber:

Definição

Mede a quantidade de caminhos linearmente independentes em um módulo de programa.

Fórmula

$$M = E - N + 2P \quad \boxed{M = E - N + 2P}$$

Para uma única função:

$$M = E - N + 2 \quad \boxed{M = E - N + 2}$$

Regra prática

Para casos simples:

$$M = \text{número de decisões} + 1 \quad \boxed{M = \text{número de decisões} + 1}$$

Interpretação

Maior complexidade tende a significar:

- mais caminhos;
- maior esforço de teste;
- maior dificuldade de compreensão;
- maior cuidado na manutenção.

72. Limitação da métrica

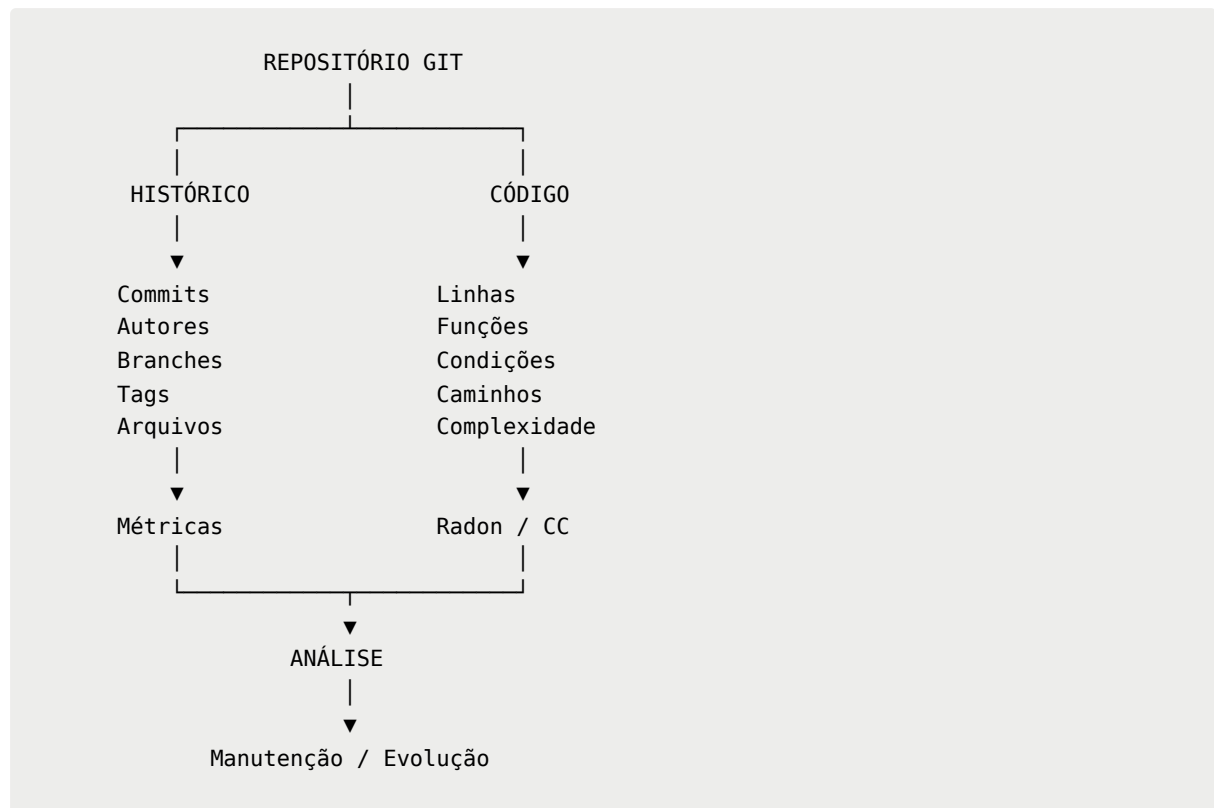
Também é importante saber:

| **Complexidade ciclomática não mede a qualidade total do software.**

Ela mede uma característica específica do fluxo de controle.

Por isso deve ser combinada com outras métricas e com análise humana.

73. Mapa mental da aula



74. Resumo final

A aula apresenta uma abordagem prática para analisar um projeto de software real.

Primeiro, o repositório é obtido com:

```
git clone
```

Depois, o histórico pode ser investigado usando comandos como:

```
git branch
git tag
```

```
git log
git shortlog
git remote
```

Esses comandos permitem obter informações sobre:

- versões;
- branches;
- contribuidores;
- commits;
- arquivos modificados;
- evolução do projeto.

Também podem ser utilizadas ferramentas de terminal para transformar esses dados em informações mais específicas, como:

- arquivos mais modificados;
- arquivos maiores;
- atividade por período;
- atividade por autor.

Depois, a análise passa do histórico para o código.

A **Complexidade Ciclomática**, proposta por Thomas McCabe, mede a quantidade de caminhos linearmente independentes em um módulo.

Sua fórmula é:

$$M = E - N + 2P$$

Para uma função isolada:

$$M = E - N + 2$$

Na prática, em exemplos simples:

$$M = \text{decisões} + 1$$

Ferramentas como o **Radon** permitem automatizar essa análise.

A principal lição da aula é:

Um projeto de software pode ser estudado a partir de dados do seu histórico e de métricas do seu código, permitindo identificar padrões,

pontos de atenção e características que ajudam nas decisões de manutenção e evolução.

E existe uma segunda lição ainda mais importante:

Métrica não é sinônimo de qualidade. Uma métrica fornece evidência; a interpretação do engenheiro é que transforma essa evidência em conhecimento.